

SENTINEL PRIME

Real-Time Fraud Detection for Digital Lending

Sudhanshu Kulkarni - 2023A7PS0414G
BITS Pilani - Goa Campus

Document scope

- System architecture and component responsibilities
- Runtime decision flow and fraud rules
- Feature schema and ML pipeline
- REST API reference and case workflow
- Local setup, Docker deployment

1. Executive Summary

Sentinel Prime Lending Risk is a real-time fraud detection prototype for digital lending. A React/TypeScript operations interface sends lending and behavioral signals to a FastAPI backend. The backend builds a versioned feature vector, evaluates deterministic fraud rules, invokes a supervised Random Forest and an Isolation Forest, combines the evidence into a bounded risk score, and applies a graduated decision policy: APPROVE, STEP_UP, MANUAL_REVIEW, or DECLINE.

Risky decisions are persisted as analyst cases. Analysts can close cases with a controlled outcome, creating a feedback trail suitable for future offline model development. The prototype supports either in-memory storage for tests or SQLite for a persistent local demonstration.

4	6	2	19
Decision actions	Live features	ML components	Backend tests passing

What is implemented

- React decision simulator and fraud operations queue.
- FastAPI REST API with request validation
- Rules for application velocity, new-device/identity mismatch, shared devices, shared payment instruments, and risky networks.
- Random Forest supervised model plus Isolation Forest anomaly evidence.
- Stable feature schema and model-version compatibility checks.
- In-memory and SQLite persistence for events, decisions, cases, and analyst outcomes.
- Docker Compose packaging for a two-service local deployment.

2. System Architecture

The implemented architecture is kept intentionally compact: the frontend is responsible for interaction and presentation, while all fraud logic remains server-side. This prevents a browser/client from supplying its own risk score or decision.

React / TypeScript UI Decision simulator - Case queue	FastAPI REST API Validation - Orchestration - Audit	Persistence In-memory or SQLite
Feature Builder features-1.0.0 - 6 canonical features	Risk Evidence Random Forest + Isolation Forest	Case Workflow MANUAL_REVIEW / DECLINE → case
Rules Engine 5 rule families	Decision Policy risk + rule evidence → action	Metrics decision and case counts

Component responsibilities

Component	Responsibility
Frontend	Collects signals, calls APIs, renders decision/risk/reasons, refreshes and resolves cases.
FastAPI	Validates requests, builds features, evaluates rules/models, applies policy, persists results.
Feature builder	Maps request fields to the canonical six-feature vector and schema version.
Rules engine	Produces deterministic reason codes and a hard-decline flag for risky networks.
Model service	Loads the versioned joblib artifact, validates schema compatibility, produces supervised/anomaly risk.
Decision policy	Combines rule evidence and weighted risk into one of four actions.
Storage	Persists events, decisions, cases, timestamps, reason codes, model/rule/schema versions.

3. Runtime Decision Flow

A scoring request follows a fixed sequence.

#	Stage	Implementation behavior
1	Request	POST /api/v1/fraud/score with six signal fields.
2	Validate	Pydantic validates ranges and requires feature schema features-1.0.0.
3	Build	Feature builder converts the request into the canonical feature vector.
4	Rules	Deterministic rules emit reason codes and may set hard_decline.
5	Models	Random Forest returns supervised risk; Isolation Forest contributes anomaly risk.
6	Combine	$\text{combined_risk} = 0.60 \cdot \text{supervised} + 0.20 \cdot \text{anomaly} + 0.20 \cdot \text{graph}$.
7	Decide	Policy maps risk and rule evidence to APPROVE / STEP_UP / MANUAL_REVIEW / DECLINE.
8	Persist	Decision audit metadata is stored; review/decline creates an open case.

Decision policy

Condition	Action
hard_decline is true OR combined_risk ≥ 0.85	DECLINE
combined_risk ≥ 0.65 OR any rule reason code exists	MANUAL_REVIEW
combined_risk ≥ 0.35	STEP_UP
otherwise	APPROVE

Important implementation detail

A rule reason code can force MANUAL_REVIEW even when the numerical model risk is below 0.65. A risky-network rule is stronger still: ip_risk_score ≥ 0.90 sets hard_decline and therefore produces DECLINE.

4. Fraud Signals, Rules & Explainability

The prototype exposes six features. They are shared by dataset generation, training, inference, and API validation.

Feature	Type / range	Meaning
applications_last_24h	integer ≥ 0	Application velocity in the preceding 24 hours.
is_new_device	boolean	Whether the current device is newly observed.
identity_match_score	0.0–1.0	Identity similarity / verification signal.
device_account_count	integer ≥ 1	Number of accounts associated with the device.
payment_account_count	integer ≥ 1	Number of accounts associated with the payment instrument.
ip_risk_score	0.0–1.0	Network/IP risk signal.

Deterministic rule catalog

Rule code	Trigger	Effect
HIGH APPLICATION VELOCITY	applications_last_24h ≥ 5	Adds reason code.
NEW DEVICE IDENTITY MISMATCH	new device AND identity score < 0.80	Adds reason code.
SHARED DEVICE	device_account_count ≥ 4	Adds reason code.
SHARED PAYMENT INSTRUMENT	payment_account_count ≥ 4	Adds reason code.
KNOWN RISKY NETWORK	ip_risk_score ≥ 0.90	Adds reason code + hard decline.

Explainability and audit fields

- Reason codes are stable identifiers rather than free-form explanations.
- Each decision records a rule version, feature schema version, model version, fallback flag, correlation ID, and decision ID.
- The frontend displays the returned evidence; it does not independently recalculate risk.
- Cases retain their reason codes when analysts close them, preserving the evidence trail.

5. Machine-Learning Pipeline

A Random Forest provides supervised fraud probability, while an Isolation Forest supplies anomaly evidence. The two signals are combined with a reserved graph-risk component.

Training data (Synthetically generated)

- 500 chronological synthetic records generated with seed 42.
- Events span hourly timestamps beginning 2026-01-01T00:00:00Z.
- Fraud labels are generated directly from the same rule families used by the runtime rules engine.
- This label construction is useful for demonstrating the pipeline, but it introduces a strong limitation: evaluation is not independent of the rules.

Training split and artifacts

Item	Value
Dataset	ml/data/fraud_events.csv
Rows	500
Training rows	400
Held-out test rows	50
Classifier	RandomForestClassifier — 150 trees, class_weight=balanced, random_state=42
Anomaly detector	IsolationForest — 100 trees, random_state=42
Decision threshold	0.50 for classifier probability
Model artifact	ml/artifacts/fraud_models.joblib
Evaluation artifact	ml/evaluation/model_metrics.json

Reported prototype metrics

97.67%
Precision

100%
Recall

1.00
PR-AUC

0.876
Synthetic fraud rate

How to interpret these metrics

The labels are synthetic and rule-derived, and the dataset is heavily fraud-positive (87.6%). Therefore the precision/recall/PR-AUC values are evidence that the training and inference pipeline works on this synthetic benchmark—not evidence of real-world fraud detection quality. Production validation requires representative, governed data, temporal evaluation, calibration, false-positive analysis, and fairness testing.

6. REST API Reference

Base URL for local development: **http://127.0.0.1:8000**

Method	Endpoint	Purpose
GET	/health	Service health check.
POST	/api/v1/events	Ingest an event; duplicate event IDs are detected.
POST	/api/v1/fraud/score	Score lending signals and return decision + audit metadata.
GET	/api/v1/decisions/{decision_id}	Retrieve a persisted decision.
GET	/api/v1/cases	List analyst review cases.
PATCH	/api/v1/cases/{case_id}	Close/update a case with a valid outcome.
GET	/api/v1/metrics	Return decision and case counts.

Example scoring request

```
{
  "correlation_id": "demo-001",
  "feature_schema_version": "features-1.0.0",
  "applications_last_24h": 1,
  "is_new_device": false,
  "identity_match_score": 0.96,
  "device_account_count": 1,
  "payment_account_count": 1,
  "ip_risk_score": 0.08
}
```

Example response shape

```
{
  "decision_id": "...",
  "correlation_id": "demo-001",
  "decision": "APPROVE",
  "combined_risk": 0.12,
  "reason_codes": [],
  "rule_version": "rules-0.1.0",
  "feature_schema_version": "features-1.0.0",
  "model_version": "fraud-model-0.1.0",
  "model_fallback": false
}
```

7. Persistence, Deployment & Configuration

Persistence boundary

Mode	Use	Behavior
InMemoryStore	Tests / quick development	Data exists only for the running process.
SQLiteStore	Local demo	Persists events, decisions, and fraud cases in SQLite.

SQLite tables

- **events** — event ID, type, customer/application references, correlation ID, timestamp.
- **decisions** — decision ID, action, combined risk, reason codes, rule/model/schema metadata, fallback flag.
- **fraud_cases** — case ID, linked decision, status, outcome, reason codes, creation/update timestamps.

Docker Compose topology

Service	Container behavior	Port
backend	FastAPI application; healthchecked; persistent volume mounted at /data	8000
frontend	Vite-built React app served by the frontend image	5173

Environment / configuration

Variable	Default / example	Purpose
FRAUD_STORE	memory	Selects memory or SQLite persistence.
FRAUD_DB_PATH	fraud_detection.db	SQLite database path when SQLite mode is enabled.
CORS_ORIGINS	localhost:5173,127.0.0.1:5173	Allowed frontend origins.
VITE_API_URL	http://127.0.0.1:8000	Frontend API base URL.

8. Local Setup

Prerequisites

- Python environment with the backend dependencies installed.
- Node.js/npm for the frontend.
- Optional: Docker and Docker Compose for containerized execution.

Backend

```
python -m pip install -r backend/requirements.txt
python -m ml.data.generate_dataset
python -m ml.training.train_models
uvicorn backend.app.main:app --reload
```

Frontend

```
cd frontend
npm install
npm run dev
```

Tests

```
python -m pytest backend/tests -q
```

Docker

```
docker compose up --build
```

Repository structure

```
frontend/
|-- src/App.tsx
|-- src/styles.css
'-- package.json

backend/app/
|-- main.py
|-- model_service.py
|-- storage.py
|-- features/builder.py
'-- domain/{decision.py, rules.py}

ml/
|-- data/{fraud_events.csv, generate_dataset.py}
|-- training/train_models.py
|-- artifacts/fraud_models.joblib
'-- evaluation/model_metrics.json
```

9. Appendix — Quick Reference

Item	Value
Project	Sentinel Prime Lending Risk
Frontend	React + TypeScript + Vite + lucide-react
Backend	FastAPI + Pydantic
ML	Random Forest + Isolation Forest
Feature schema	features-1.0.0
Model version	fraud-model-0.1.0
Rule version	rules-0.1.0
Storage	In-memory / SQLite
Default frontend	http://127.0.0.1:5173/
Default backend	http://127.0.0.1:8000/
Swagger/OpenAPI	http://127.0.0.1:8000/docs
Validation	19 backend tests passed in repository review